

# Big Data y Machine Learning para Economía Aplicada

Week 08: Redes Neuronales y Visión por Computadora

Eduard F. Martinez Gonzalez, Ph.D.

Departamento de Economía, Universidad Icesi

July 23, 2026

## Previously...

La frase con la que cerró la sesión pasada: **representar es comprimir con sentido** — y PCA lo hacía *linealmente*.

## Previously...

La frase con la que cerró la sesión pasada: **representar es comprimir con sentido** — y PCA lo hacía *linealmente*.

Hoy convergen todas las piezas que hemos ido sembrando:

- El **gradiente** como motor de aprendizaje (*boosting*, S4; verosimilitud logística, S5).
- La **regularización** como precio de la flexibilidad (S2–S3).
- La **sigmoide** y la *cross-entropy* (S5).
- Las **representaciones** aprendidas (S7).

## Previously...

La frase con la que cerró la sesión pasada: **representar es comprimir con sentido** — y PCA lo hacía *linealmente*.

Hoy convergen todas las piezas que hemos ido sembrando:

- El **gradiente** como motor de aprendizaje (*boosting*, S4; verosimilitud logística, S5).
- La **regularización** como precio de la flexibilidad (S2–S3).
- La **sigmoide** y la *cross-entropy* (S5).
- Las **representaciones** aprendidas (S7).

## Hoy: el aproximador universal

- 1 **Redes feedforward**: qué son, por qué aproximan cualquier función, y cómo se entrenan (**retropropagación**, derivada en tablero: va al Taller 2).
- 2 **Redes convolucionales**: la arquitectura que lee imágenes.
- 3 **Economía desde el cielo**: predecir pobreza con imágenes satelitales.

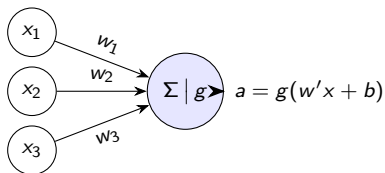
- 1 Del perceptrón a la red
- 2 Entrenamiento: SGD y retropropagación
- 3 Redes convolucionales: visión
- 4 Economía desde el cielo

# La neurona: una regresión con actitud

La unidad básica no tiene ningún misterio:

$$a = g(w'x + b)$$

combinación lineal de las entradas + una **función de activación** no lineal  $g$ .

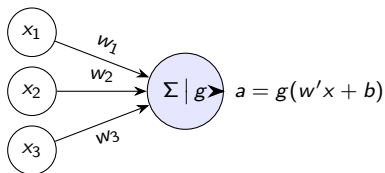


# La neurona: una regresión con actitud

La unidad básica no tiene ningún misterio:

$$a = g(w'x + b)$$

combinación lineal de las entradas + una **función de activación** no lineal  $g$ .



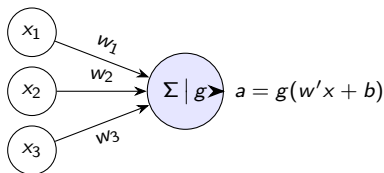
- Con  $g$  sigmoide, una neurona **es** una regresión logística (S5). Nada nuevo aún.
- La novedad no es la neurona: es **componerlas**.

# La neurona: una regresión con actitud

La unidad básica no tiene ningún misterio:

$$a = g(w'x + b)$$

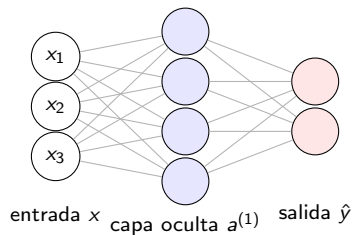
combinación lineal de las entradas + una **función de activación** no lineal  $g$ .



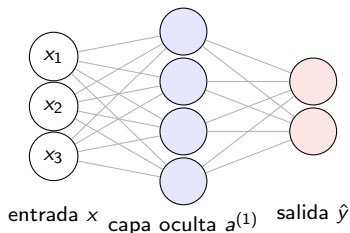
- Con  $g$  sigmoide, una neurona **es** una regresión logística (S5). Nada nuevo aún.
- La novedad no es la neurona: es **componerlas**.
- Sin la no linealidad no hay negocio: componer funciones lineales da... otra función lineal.  $g$  es lo que hace que apilar capas agregue capacidad.



# La red feedforward: composición de capas



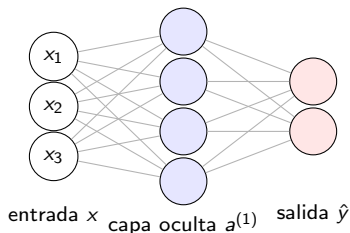
## La red feedforward: composición de capas



**En notación matricial, capa por capa (con  $a^{(0)} = x$ ):**

$$a^{(l)} = g(W^{(l)}a^{(l-1)} + b^{(l)}), \quad l = 1, \dots, L-1; \quad \hat{y} = h(W^{(L)}a^{(L-1)} + b^{(L)})$$

# La red feedforward: composición de capas



**En notación matricial, capa por capa** (con  $a^{(0)} = x$ ):

$$a^{(l)} = g(W^{(l)}a^{(l-1)} + b^{(l)}), \quad l = 1, \dots, L-1; \quad \hat{y} = h(W^{(L)}a^{(L-1)} + b^{(L)})$$

- La red **es** una composición:  $\hat{y} = f_L \circ \dots \circ f_1(x)$ . Profundidad = cuántas funciones componemos.
- Los “parámetros” son todos los  $W, b$ : fácilmente miles o millones — flexibilidad brutal que exigirá regularización brutal (lo sabíamos desde la S1).
- Salida  $h$  según la tarea: identidad (regresión), sigmoide (binaria), **softmax** ( $K$  clases).

# ¿Por qué tanta fe? Aproximación universal y profundidad

**Teorema de aproximación universal** (Cybenko, 1989; Hornik, 1991): una red con **una** capa oculta suficientemente ancha aproxima cualquier función continua tan bien como se quiera.

# ¿Por qué tanta fe? Aproximación universal y profundidad

**Teorema de aproximación universal** (Cybenko, 1989; Hornik, 1991): una red con **una** capa oculta suficientemente ancha aproxima cualquier función continua tan bien como se quiera.

- Suena definitivo, pero es una garantía de *existencia*: no dice cuántas neuronas (pueden ser astronómicas) ni cómo encontrar los pesos.

# ¿Por qué tanta fe? Aproximación universal y profundidad

**Teorema de aproximación universal** (Cybenko, 1989; Hornik, 1991): una red con **una** capa oculta suficientemente ancha aproxima cualquier función continua tan bien como se quiera.

- Suena definitivo, pero es una garantía de *existencia*: no dice cuántas neuronas (pueden ser astronómicas) ni cómo encontrar los pesos.
- **¿Por qué profundo y no ancho, entonces?** La composición **reutiliza**: las capas tempranas aprenden piezas (bordes, patrones básicos) que las tardías *combinan*. Funciones con estructura composicional se representan con muchos menos parámetros en profundidad que en anchura.

# ¿Por qué tanta fe? Aproximación universal y profundidad

**Teorema de aproximación universal** (Cybenko, 1989; Hornik, 1991): una red con **una** capa oculta suficientemente ancha aproxima cualquier función continua tan bien como se quiera.

- Suena definitivo, pero es una garantía de *existencia*: no dice cuántas neuronas (pueden ser astronómicas) ni cómo encontrar los pesos.
- **¿Por qué profundo y no ancho, entonces?** La composición **reutiliza**: las capas tempranas aprenden piezas (bordes, patrones básicos) que las tardías *combinan*. Funciones con estructura composicional se representan con muchos menos parámetros en profundidad que en anchura.
- Y la lectura que conecta con la sesión 7: **cada capa oculta es una representación aprendida** de los datos — de cruda a abstracta. La red no solo predice: fabrica las variables con las que predice.

# ¿Por qué tanta fe? Aproximación universal y profundidad

**Teorema de aproximación universal** (Cybenko, 1989; Hornik, 1991): una red con **una** capa oculta suficientemente ancha aproxima cualquier función continua tan bien como se quiera.

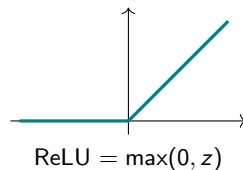
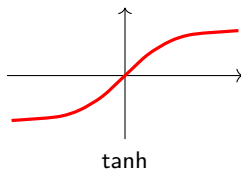
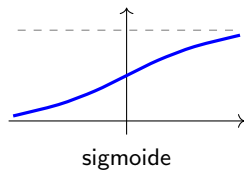
- Suena definitivo, pero es una garantía de *existencia*: no dice cuántas neuronas (pueden ser astronómicas) ni cómo encontrar los pesos.
- **¿Por qué profundo y no ancho, entonces?** La composición **reutiliza**: las capas tempranas aprenden piezas (bordes, patrones básicos) que las tardías *combinan*. Funciones con estructura composicional se representan con muchos menos parámetros en profundidad que en anchura.
- Y la lectura que conecta con la sesión 7: **cada capa oculta es una representación aprendida** de los datos — de cruda a abstracta. La red no solo predice: fabrica las variables con las que predice.

## El contrato de la flexibilidad

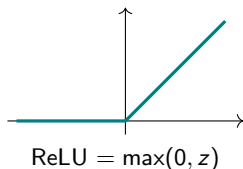
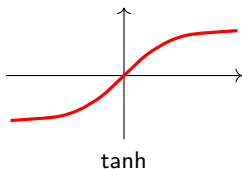
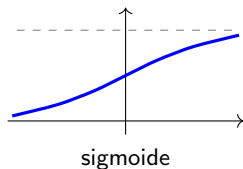
Aproximarlo todo  $\Rightarrow$  sobreajustarlo todo. El teorema garantiza capacidad; la generalización se compra con datos y regularización — la curva U jamás se fue.



# Funciones de activación



# Funciones de activación



- **Sigmoide/tanh**: históricas y elegantes, pero se **saturan**: en las colas su derivada es  $\approx 0$ , y con muchas capas el gradiente se *desvanece* multiplicándose por casi-ceros.
- **ReLU**: tosca (¡ni siquiera es diferenciable en 0!) pero no se satura en la mitad positiva, es baratísima, y en la práctica entrena mucho mejor: el **estándar** en capas ocultas.
- **Softmax** en la salida multiclase: convierte  $K$  puntajes en probabilidades ( $e^{z_k} / \sum_j e^{z_j}$ ) — la generalización de la logística que ya conocen.

- 1 Del perceptrón a la red
- 2 Entrenamiento: SGD y retropropagación**
- 3 Redes convolucionales: visión
- 4 Economía desde el cielo

# La pérdida, y una mala noticia asumida con calma

## Nada nuevo en la pérdida:

- Regresión:  $L = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$  (el MSE de siempre).
- Clasificación:  $cross-entropy = -\frac{1}{n} \sum_i \sum_k y_{ik} \log \hat{p}_{ik}$  — que es, literalmente, la **máxima verosimilitud** de la sesión 5 con signo menos.

# La pérdida, y una mala noticia asumida con calma

## Nada nuevo en la pérdida:

- Regresión:  $L = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$  (el MSE de siempre).
- Clasificación:  $\text{cross-entropy} = -\frac{1}{n} \sum_i \sum_k y_{ik} \log \hat{p}_{ik}$  — que es, literalmente, la **máxima verosimilitud** de la sesión 5 con signo menos.

**La mala noticia:** como función de los pesos  $(W, b)$ , la pérdida **no es convexa**: paisaje lleno de valles, mesetas y puntos de silla. Adiós garantías globales tipo Lasso.

# La pérdida, y una mala noticia asumida con calma

## Nada nuevo en la pérdida:

- Regresión:  $L = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$  (el MSE de siempre).
- Clasificación:  $cross-entropy = -\frac{1}{n} \sum_i \sum_k y_{ik} \log \hat{p}_{ik}$  — que es, literalmente, la **máxima verosimilitud** de la sesión 5 con signo menos.

**La mala noticia:** como función de los pesos  $(W, b)$ , la pérdida **no es convexa**: paisaje lleno de valles, mesetas y puntos de silla. Adiós garantías globales tipo Lasso.

**La sorpresa empírica que sostiene todo el campo:** el descenso de gradiente *estocástico*, sin garantía teórica de óptimo global, encuentra **sistemáticamente** soluciones que generalizan bien en redes grandes. Los mínimos “suficientemente buenos” abundan.

# La pérdida, y una mala noticia asumida con calma

## Nada nuevo en la pérdida:

- Regresión:  $L = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$  (el MSE de siempre).
- Clasificación:  $\text{cross-entropy} = -\frac{1}{n} \sum_i \sum_k y_{ik} \log \hat{p}_{ik}$  — que es, literalmente, la **máxima verosimilitud** de la sesión 5 con signo menos.

**La mala noticia:** como función de los pesos  $(W, b)$ , la pérdida **no es convexa**: paisaje lleno de valles, mesetas y puntos de silla. Adiós garantías globales tipo Lasso.

**La sorpresa empírica que sostiene todo el campo:** el descenso de gradiente *estocástico*, sin garantía teórica de óptimo global, encuentra **sistemáticamente** soluciones que generalizan bien en redes grandes. Los mínimos “suficientemente buenos” abundan.

## Actitud del curso

Igual que con  $k$ -means (S7): óptimo local + buen protocolo (inicializaciones, monitoreo, regularización) = herramienta utilizable. La perfección teórica no es requisito para la utilidad práctica.

# Descenso de gradiente estocástico (SGD)

**La regla de siempre**, ahora sobre millones de parámetros  $\theta = (W, b)$ :

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta)$$



# Descenso de gradiente estocástico (SGD)

La regla de **siempre**, ahora sobre millones de parámetros  $\theta = (W, b)$ :

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta)$$

Las tres variantes:

- **Batch**: gradiente con *todos* los datos por paso — exacto y carísimo.
- **Estocástico puro**: una observación por paso — barato y ruidosísimo.
- **Mini-batch** (el estándar): 32–256 observaciones por paso: gradiente ruidoso pero decente, y el ruido hasta ayuda a escapar de valles malos.

# Descenso de gradiente estocástico (SGD)

La regla de **siempre**, ahora sobre millones de parámetros  $\theta = (W, b)$ :

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta)$$

## Las tres variantes:

- **Batch**: gradiente con *todos* los datos por paso — exacto y carísimo.
- **Estocástico puro**: una observación por paso — barato y ruidosísimo.
- **Mini-batch** (el estándar): 32–256 observaciones por paso: gradiente ruidoso pero decente, y el ruido hasta ayuda a escapar de valles malos.
- $\eta$  (*learning rate*): la perilla delicada — grande diverge, chica no llega. Se afina con *schedules* o con **Adam** (pasos adaptativos por parámetro; el default moderno).
- Una **época** = una pasada por los datos; se entrena por épocas monitoreando validación.

# Descenso de gradiente estocástico (SGD)

La regla de **siempre**, ahora sobre millones de parámetros  $\theta = (W, b)$ :

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta)$$

## Las tres variantes:

- **Batch**: gradiente con *todos* los datos por paso — exacto y carísimo.
- **Estocástico puro**: una observación por paso — barato y ruidosísimo.
- **Mini-batch** (el estándar): 32–256 observaciones por paso: gradiente ruidoso pero decente, y el ruido hasta ayuda a escapar de valles malos.
- $\eta$  (*learning rate*): la perilla delicada — grande diverge, chica no llega. Se afina con *schedules* o con **Adam** (pasos adaptativos por parámetro; el default moderno).
- Una **época** = una pasada por los datos; se entrena por épocas monitoreando validación.

## Deja vu del boosting

En la S4 hicimos descenso de gradiente *en el espacio de funciones* (un árbol por

# Retropropagación (I): la idea

**El problema computacional:**  $\nabla_{\theta} L$  tiene millones de componentes. ¿Cómo calcularlos sin morir?

# Retropropagación (I): la idea

**El problema computacional:**  $\nabla_{\theta} L$  tiene millones de componentes. ¿Cómo calcularlos sin morir?

**La respuesta: la regla de la cadena, administrada con inteligencia.**

- La red es una composición  $\Rightarrow$  la derivada de la pérdida respecto a *cualquier* peso es un producto de derivadas locales a lo largo del camino hasta la salida.
- La observación clave: esos productos **comparten factores**. Computando *de la salida hacia atrás*, cada factor se calcula **una sola vez** y se reutiliza.

# Retropropagación (I): la idea

**El problema computacional:**  $\nabla_{\theta} L$  tiene millones de componentes. ¿Cómo calcularlos sin morir?

**La respuesta: la regla de la cadena, administrada con inteligencia.**

- La red es una composición  $\Rightarrow$  la derivada de la pérdida respecto a *cualquier* peso es un producto de derivadas locales a lo largo del camino hasta la salida.
- La observación clave: esos productos **comparten factores**. Computando *de la salida hacia atrás*, cada factor se calcula **una sola vez** y se reutiliza.
- Resultado: el gradiente completo cuesta lo mismo que **una pasada hacia adelante + una hacia atrás** — no una pasada por parámetro. Esa economía es lo que hace entrenables las redes gigantes.

# Retropropagación (I): la idea

**El problema computacional:**  $\nabla_{\theta} L$  tiene millones de componentes. ¿Cómo calcularlos sin morir?

**La respuesta: la regla de la cadena, administrada con inteligencia.**

- La red es una composición  $\Rightarrow$  la derivada de la pérdida respecto a *cualquier* peso es un producto de derivadas locales a lo largo del camino hasta la salida.
- La observación clave: esos productos **comparten factores**. Computando *de la salida hacia atrás*, cada factor se calcula **una sola vez** y se reutiliza.
- Resultado: el gradiente completo cuesta lo mismo que **una pasada hacia adelante + una hacia atrás** — no una pasada por parámetro. Esa economía es lo que hace entrenables las redes gigantes.
- El objeto que viaja hacia atrás: el “error” de cada capa,  $\delta^{(l)} = \partial L / \partial z^{(l)}$ .
- En la práctica nadie deriva a mano: **diferenciación automática** (Keras/PyTorch). Pero ustedes sí lo harán una vez — ahora mismo — porque es material del **Taller 2**.

## Retropropagación (II): derivación con una capa oculta

**Setup** (regresión, pérdida cuadrática de una observación):

$$z = W^{(1)}x + b^{(1)}, \quad h = g(z), \quad \hat{y} = w^{(2)'}h + b^{(2)}, \quad L = \frac{1}{2}(\hat{y} - y)^2$$



## Retropropagación (II): derivación con una capa oculta

**Setup** (regresión, pérdida cuadrática de una observación):

$$z = W^{(1)}x + b^{(1)}, \quad h = g(z), \quad \hat{y} = w^{(2)'}h + b^{(2)}, \quad L = \frac{1}{2}(\hat{y} - y)^2$$

**1. Capa de salida** (regla de la cadena, primer eslabón):

$$\delta^{(2)} = \frac{\partial L}{\partial \hat{y}} = \hat{y} - y \quad (\text{¡el residuo, otra vez!})$$

## Retropropagación (II): derivación con una capa oculta

**Setup** (regresión, pérdida cuadrática de una observación):

$$z = W^{(1)}x + b^{(1)}, \quad h = g(z), \quad \hat{y} = w^{(2)'}h + b^{(2)}, \quad L = \frac{1}{2}(\hat{y} - y)^2$$

**1. Capa de salida** (regla de la cadena, primer eslabón):

$$\delta^{(2)} = \frac{\partial L}{\partial \hat{y}} = \hat{y} - y \quad (\text{¡el residuo, otra vez!})$$

**2. Gradientes de la capa de salida:**

$$\frac{\partial L}{\partial w^{(2)}} = \delta^{(2)} h, \quad \frac{\partial L}{\partial b^{(2)}} = \delta^{(2)}$$

# Retropropagación (II): derivación con una capa oculta

**Setup** (regresión, pérdida cuadrática de una observación):

$$z = W^{(1)}x + b^{(1)}, \quad h = g(z), \quad \hat{y} = w^{(2)'}h + b^{(2)}, \quad L = \frac{1}{2}(\hat{y} - y)^2$$

**1. Capa de salida** (regla de la cadena, primer eslabón):

$$\delta^{(2)} = \frac{\partial L}{\partial \hat{y}} = \hat{y} - y \quad (\text{¡el residuo, otra vez!})$$

**2. Gradientes de la capa de salida:**

$$\frac{\partial L}{\partial w^{(2)}} = \delta^{(2)} h, \quad \frac{\partial L}{\partial b^{(2)}} = \delta^{(2)}$$

**3. Propagar el error hacia atrás** (por cada neurona oculta, la cadena pasa por  $\hat{y}$  y por  $g$ ):

$$\delta^{(1)} = (w^{(2)} \delta^{(2)}) \odot g'(z)$$

# Retropropagación (II): derivación con una capa oculta

**Setup** (regresión, pérdida cuadrática de una observación):

$$z = W^{(1)}x + b^{(1)}, \quad h = g(z), \quad \hat{y} = w^{(2)'}h + b^{(2)}, \quad L = \frac{1}{2}(\hat{y} - y)^2$$

**1. Capa de salida** (regla de la cadena, primer eslabón):

$$\delta^{(2)} = \frac{\partial L}{\partial \hat{y}} = \hat{y} - y \quad (\text{¡el residuo, otra vez!})$$

**2. Gradientes de la capa de salida:**

$$\frac{\partial L}{\partial w^{(2)}} = \delta^{(2)} h, \quad \frac{\partial L}{\partial b^{(2)}} = \delta^{(2)}$$

**3. Propagar el error hacia atrás** (por cada neurona oculta, la cadena pasa por  $\hat{y}$  y por  $g$ ):

$$\delta^{(1)} = (w^{(2)} \delta^{(2)}) \odot g'(z)$$

**4. Gradientes de la capa oculta:**

$$\frac{\partial L}{\partial W^{(1)}} = \delta^{(1)} x', \quad \frac{\partial L}{\partial b^{(1)}} = \delta^{(1)}$$

# Retropropagación (II): derivación con una capa oculta

**Setup** (regresión, pérdida cuadrática de una observación):

$$z = W^{(1)}x + b^{(1)}, \quad h = g(z), \quad \hat{y} = w^{(2)'}h + b^{(2)}, \quad L = \frac{1}{2}(\hat{y} - y)^2$$

**1. Capa de salida** (regla de la cadena, primer eslabón):

$$\delta^{(2)} = \frac{\partial L}{\partial \hat{y}} = \hat{y} - y \quad (\text{¡el residuo, otra vez!})$$

**2. Gradientes de la capa de salida:**

$$\frac{\partial L}{\partial w^{(2)}} = \delta^{(2)} h, \quad \frac{\partial L}{\partial b^{(2)}} = \delta^{(2)}$$

**3. Propagar el error hacia atrás** (por cada neurona oculta, la cadena pasa por  $\hat{y}$  y por  $g$ ):

$$\delta^{(1)} = (w^{(2)} \delta^{(2)}) \odot g'(z)$$

**4. Gradientes de la capa oculta:**

$$\frac{\partial L}{\partial W^{(1)}} = \delta^{(1)} x', \quad \frac{\partial L}{\partial b^{(1)}} = \delta^{(1)}$$

## Para el Taller 2

“Un paso de retropropagación en una red simple” = estos cuatro pasos, con

# Regularizar redes: el kit completo

Millones de parámetros  $\Rightarrow$  el sobreajuste no es riesgo, es **default**. El arsenal (todo les va a sonar):

# Regularizar redes: el kit completo

Millones de parámetros  $\Rightarrow$  el sobreajuste no es riesgo, es **default**. El arsenal (todo les va a sonar):

- **Weight decay** ( $L_2$ ): penalizar  $\sum \|W\|^2$  — Ridge (S2) con otro nombre y la misma alma.
- **Early stopping**: monitorear la pérdida de *validación* y parar cuando sube — es **elegir el mínimo de la curva U en vivo** (S1–S2), con las épocas como eje de complejidad.
- **Dropout** (Srivastava et al., 2014): en cada paso, apagar al azar una fracción de neuronas  $\Rightarrow$  la red no puede depender de nadie en particular; en el fondo, un **ensamble implícito** de sub-redes (¡el espíritu del Random Forest, S4!).
- **Batch normalization**: estandarizar activaciones dentro de la red (nuestra vieja obsesión por estandarizar, ahora capa por capa); estabiliza y acelera.
- **Aumentar datos** (*augmentation*): en imágenes, rotar/recortar/reflejar — fabricar variación invariante. El regularizador más honesto: más datos.

# Regularizar redes: el kit completo

Millones de parámetros  $\Rightarrow$  el sobreajuste no es riesgo, es **default**. El arsenal (todo les va a sonar):

- **Weight decay** ( $L_2$ ): penalizar  $\sum \|W\|^2$  — Ridge (S2) con otro nombre y la misma alma.
- **Early stopping**: monitorear la pérdida de *validación* y parar cuando sube — es **elegir el mínimo de la curva U en vivo** (S1–S2), con las épocas como eje de complejidad.
- **Dropout** (Srivastava et al., 2014): en cada paso, apagar al azar una fracción de neuronas  $\Rightarrow$  la red no puede depender de nadie en particular; en el fondo, un **ensamble implícito** de sub-redes (¡el espíritu del Random Forest, S4!).
- **Batch normalization**: estandarizar activaciones dentro de la red (nuestra vieja obsesión por estandarizar, ahora capa por capa); estabiliza y acelera.
- **Aumentar datos** (*augmentation*): en imágenes, rotar/recortar/reflejar — fabricar variación invariante. El regularizador más honesto: más datos.

## Cuándo NO usar redes

Datos tabulares medianos: XGBoost/RF suelen ganar con una fracción del



- 1 Del perceptrón a la red
- 2 Entrenamiento: SGD y retropropagación
- 3 Redes convolucionales: visión
- 4 Economía desde el cielo

# Por qué una red densa no puede “ver”

Una imagen modesta de  $200 \times 200$  píxeles a color = 120.000 entradas.

# Por qué una red densa no puede “ver”

Una imagen modesta de  $200 \times 200$  píxeles a color = 120.000 entradas.

- Primera capa densa de 1.000 neuronas  $\Rightarrow$  **120 millones** de pesos. . . en una capa. Sobreajuste garantizado y cómputo absurdo.

# Por qué una red densa no puede “ver”

Una imagen modesta de  $200 \times 200$  píxeles a color = 120.000 entradas.

- Primera capa densa de 1.000 neuronas  $\Rightarrow$  **120 millones** de pesos. . . en una capa. Sobreajuste garantizado y cómputo absurdo.
- Peor: la red densa **ignora la estructura espacial**. Trata el píxel (1,1) y el (200,200) como si su relación fuera igual a la de dos vecinos — y si el gato se corre 10 píxeles, para la red es una entrada completamente nueva.

# Por qué una red densa no puede “ver”

Una imagen modesta de  $200 \times 200$  píxeles a color = 120.000 entradas.

- Primera capa densa de 1.000 neuronas  $\Rightarrow$  **120 millones** de pesos... en una capa. Sobreajuste garantizado y cómputo absurdo.
- Peor: la red densa **ignora la estructura espacial**. Trata el píxel (1,1) y el (200,200) como si su relación fuera igual a la de dos vecinos — y si el gato se corre 10 píxeles, para la red es una entrada completamente nueva.

## Las dos ideas que arreglan todo (LeCun, 1989):

- 1 **Localidad**: los patrones visuales son locales (un borde vive en una ventanita, no en toda la imagen).
- 2 **Invarianza a traslación**: un borde es un borde *esté donde esté*  $\Rightarrow$  usar **los mismos pesos** en toda la imagen.

# Por qué una red densa no puede “ver”

Una imagen modesta de  $200 \times 200$  píxeles a color = 120.000 entradas.

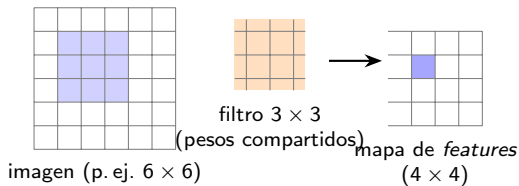
- Primera capa densa de 1.000 neuronas  $\Rightarrow$  **120 millones** de pesos... en una capa. Sobreajuste garantizado y cómputo absurdo.
- Peor: la red densa **ignora la estructura espacial**. Trata el píxel (1,1) y el (200,200) como si su relación fuera igual a la de dos vecinos — y si el gato se corre 10 píxeles, para la red es una entrada completamente nueva.

## Las dos ideas que arreglan todo (LeCun, 1989):

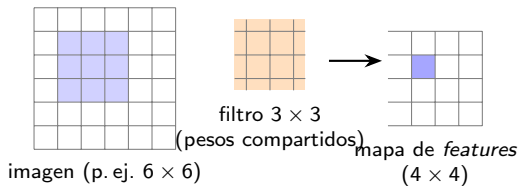
- 1 **Localidad**: los patrones visuales son locales (un borde vive en una ventanita, no en toda la imagen).
- 2 **Invarianza a traslación**: un borde es un borde *esté donde esté*  $\Rightarrow$  usar **los mismos pesos** en toda la imagen.

Juntas se llaman **convolución** — y convierten los 120 millones de pesos en unas decenas por filtro.

# La convolución: un filtro que recorre la imagen



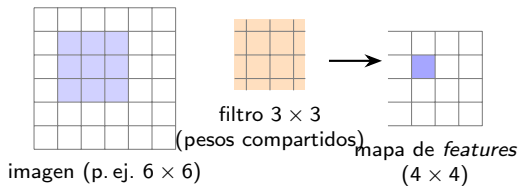
# La convolución: un filtro que recorre la imagen



- El filtro (una matriz chica de pesos) se **desliza** por la imagen; en cada posición: producto punto ventana–filtro  $\rightarrow$  un número del mapa de salida.
- Cada filtro aprende a **detectar un patrón** (un borde vertical, una esquina, una textura); su mapa dice *dónde* apareció.
- Una capa convolucional = varios filtros en paralelo; los pesos a aprender son los filtros — decenas, no millones.



# La convolución: un filtro que recorre la imagen



- El filtro (una matriz chica de pesos) se **desliza** por la imagen; en cada posición: producto punto ventana–filtro → un número del mapa de salida.
- Cada filtro aprende a **detectar un patrón** (un borde vertical, una esquina, una textura); su mapa dice *dónde* apareció.
- Una capa convolucional = varios filtros en paralelo; los pesos a aprender son los filtros — decenas, no millones.
- **Pooling** (submuestreo, p. ej. el máximo de cada 2 × 2): reduce resolución, gana robustez a desplazamientos pequeños y abarata lo que sigue.

# La arquitectura CNN: jerarquía de representaciones

**El patrón que se repite:** [ convolución  $\rightarrow$  ReLU  $\rightarrow$  pooling ]  $\times$  varias veces  $\rightarrow$  capas densas  $\rightarrow$  softmax.

# La arquitectura CNN: jerarquía de representaciones

**El patrón que se repite:** [ convolución  $\rightarrow$  ReLU  $\rightarrow$  pooling ]  $\times$  varias veces  $\rightarrow$  capas densas  $\rightarrow$  softmax.

**Lo que aprende cada nivel** (verificado mirando dentro de redes entrenadas):

- Capas tempranas: **bordes** y gradientes de color.
- Capas medias: **texturas y partes** (ventanas, ruedas, techos, esquinas).
- Capas tardías: **objetos y escenas** (una casa, una vía, un cultivo).

La **composición** de la Parte 1, hecha visible: representaciones de lo simple a lo abstracto.

# La arquitectura CNN: jerarquía de representaciones

**El patrón que se repite:** [ convolución  $\rightarrow$  ReLU  $\rightarrow$  pooling ]  $\times$  varias veces  $\rightarrow$  capas densas  $\rightarrow$  softmax.

**Lo que aprende cada nivel** (verificado mirando dentro de redes entrenadas):

- Capas tempranas: **bordes** y gradientes de color.
- Capas medias: **texturas y partes** (ventanas, ruedas, techos, esquinas).
- Capas tardías: **objetos y escenas** (una casa, una vía, un cultivo).

La **composición** de la Parte 1, hecha visible: representaciones de lo simple a lo abstracto.

**El momento histórico:** AlexNet gana ImageNet (2012) por paliza — profundidad + ReLU + dropout + GPUs. Desde entonces, la visión por computadora **es** aprendizaje profundo.

# La arquitectura CNN: jerarquía de representaciones

**El patrón que se repite:** [ convolución  $\rightarrow$  ReLU  $\rightarrow$  pooling ]  $\times$  varias veces  $\rightarrow$  capas densas  $\rightarrow$  softmax.

**Lo que aprende cada nivel** (verificado mirando dentro de redes entrenadas):

- Capas tempranas: **bordes** y gradientes de color.
- Capas medias: **texturas y partes** (ventanas, ruedas, techos, esquinas).
- Capas tardías: **objetos y escenas** (una casa, una vía, un cultivo).

La **composición** de la Parte 1, hecha visible: representaciones de lo simple a lo abstracto.

**El momento histórico:** AlexNet gana ImageNet (2012) por paliza — profundidad + ReLU + dropout + GPUs. Desde entonces, la visión por computadora **es** aprendizaje profundo.

## Transfer learning: el truco que nos habilita a nosotros

Una CNN entrenada en millones de imágenes genéricas ya sabe ver bordes, texturas y partes — eso es **universal**. Receta: tomar la red preentrenada, **congelar** las capas tempranas y reentrenar solo las finales con *nuestros* datos (miles, no millones). Así se hace visión con presupuesto de economista.

- 1 Del perceptrón a la red
- 2 Entrenamiento: SGD y retropropagación
- 3 Redes convolucionales: visión
- 4 Economía desde el cielo

## Jean et al. (2016, Science): pobreza vista desde satélites

**El problema:** en buena parte del mundo en desarrollo, las encuestas de hogares son escasas, caras y poco frecuentes — justo donde más se necesita medir pobreza.

## Jean et al. (2016, Science): pobreza vista desde satélites

**El problema:** en buena parte del mundo en desarrollo, las encuestas de hogares son escasas, caras y poco frecuentes — justo donde más se necesita medir pobreza.

**La solución, paso a paso** (transfer learning doblemente inteligente):

- 1 Partir de una CNN **preentrenada** en imágenes genéricas (ImageNet).
- 2 Reentrenarla para predecir **luces nocturnas** a partir de imágenes satelitales **diurnas** — las luces son un proxy conocido de actividad económica, y hay datos de sobra (no se necesita ni una encuesta para este paso).
- 3 En ese proceso, la red aprende sola a reconocer techos, vías, cultivos, urbanización. . .
- 4 Usar esas *features* para predecir el **consumo medido en las encuestas** disponibles.



## Jean et al. (2016, Science): pobreza vista desde satélites

**El problema:** en buena parte del mundo en desarrollo, las encuestas de hogares son escasas, caras y poco frecuentes — justo donde más se necesita medir pobreza.

**La solución, paso a paso** (transfer learning doblemente inteligente):

- 1 Partir de una CNN **preentrenada** en imágenes genéricas (ImageNet).
- 2 Reentrenarla para predecir **luces nocturnas** a partir de imágenes satelitales **diurnas** — las luces son un proxy conocido de actividad económica, y hay datos de sobra (no se necesita ni una encuesta para este paso).
- 3 En ese proceso, la red aprende sola a reconocer techos, vías, cultivos, urbanización. . .
- 4 Usar esas *features* para predecir el **consumo medido en las encuestas** disponibles.

**Resultado:** explica entre **37% y 55%** de la variación del consumo promedio local en cinco países africanos — con datos esencialmente **gratis y actualizables**.

## Jean et al. (2016, Science): pobreza vista desde satélites

**El problema:** en buena parte del mundo en desarrollo, las encuestas de hogares son escasas, caras y poco frecuentes — justo donde más se necesita medir pobreza.

**La solución, paso a paso** (transfer learning doblemente inteligente):

- 1 Partir de una CNN **preentrenada** en imágenes genéricas (ImageNet).
- 2 Reentrenarla para predecir **luces nocturnas** a partir de imágenes satelitales **diurnas** — las luces son un proxy conocido de actividad económica, y hay datos de sobra (no se necesita ni una encuesta para este paso).
- 3 En ese proceso, la red aprende sola a reconocer techos, vías, cultivos, urbanización...
- 4 Usar esas *features* para predecir el **consumo medido en las encuestas** disponibles.

**Resultado:** explica entre **37% y 55%** de la variación del consumo promedio local en cinco países africanos — con datos esencialmente **gratis y actualizables**.

Por qué es la aplicación perfecta del curso

Transfer learning (hoy) + proxies y validación *downstream* (S7) + evaluación

# El ecosistema: luces, píxeles y política pública

## La familia de aplicaciones que abre esta caja:

- **Luces nocturnas como PIB local** (Henderson, Storeygard & Weil, 2012, AER): medir crecimiento donde las cuentas nacionales son débiles o dudosas; hoy, estándar para PIB municipal y conflicto.

# El ecosistema: luces, píxeles y política pública

## La familia de aplicaciones que abre esta caja:

- **Luces nocturnas como PIB local** (Henderson, Storeygard & Weil, 2012, AER): medir crecimiento donde las cuentas nacionales son débiles o dudosas; hoy, estándar para PIB municipal y conflicto.
- **Deforestación y uso del suelo**: monitoreo casi en tiempo real con clasificación de píxeles — la base de la fiscalización ambiental moderna (Amazonía incluida).

# El ecosistema: luces, píxeles y política pública

## La familia de aplicaciones que abre esta caja:

- **Luces nocturnas como PIB local** (Henderson, Storeygard & Weil, 2012, AER): medir crecimiento donde las cuentas nacionales son débiles o dudosas; hoy, estándar para PIB municipal y conflicto.
- **Deforestación y uso del suelo**: monitoreo casi en tiempo real con clasificación de píxeles — la base de la fiscalización ambiental moderna (Amazonía incluida).
- **Urbanización y vivienda**: detectar construcciones, medir expansión informal, actualizar catastros — con obvia resonancia colombiana.

# El ecosistema: luces, píxeles y política pública

## La familia de aplicaciones que abre esta caja:

- **Luces nocturnas como PIB local** (Henderson, Storeygard & Weil, 2012, AER): medir crecimiento donde las cuentas nacionales son débiles o dudosas; hoy, estándar para PIB municipal y conflicto.
- **Deforestación y uso del suelo**: monitoreo casi en tiempo real con clasificación de píxeles — la base de la fiscalización ambiental moderna (Amazonía incluida).
- **Urbanización y vivienda**: detectar construcciones, medir expansión informal, actualizar catastros — con obvia resonancia colombiana.
- **Focalización**: mapas de pobreza de alta resolución para asignar programas donde las encuestas no llegan.

# El ecosistema: luces, píxeles y política pública

## La familia de aplicaciones que abre esta caja:

- **Luces nocturnas como PIB local** (Henderson, Storeygard & Weil, 2012, AER): medir crecimiento donde las cuentas nacionales son débiles o dudosas; hoy, estándar para PIB municipal y conflicto.
- **Deforestación y uso del suelo**: monitoreo casi en tiempo real con clasificación de píxeles — la base de la fiscalización ambiental moderna (Amazonía incluida).
- **Urbanización y vivienda**: detectar construcciones, medir expansión informal, actualizar catastros — con obvia resonancia colombiana.
- **Focalización**: mapas de pobreza de alta resolución para asignar programas donde las encuestas no llegan.

## La advertencia de medición (siempre la hay)

Lo predicho es un **proxy**: hereda los sesgos de su entrenamiento (¿qué pobreza *no* se ve desde arriba?) y se degrada fuera del dominio (otro país, otra década). Validar contra terreno — la lección de la S7 — no es opcional.

# Recap

- 1 Una neurona es una logística; la potencia está en la **composición**: capas = representaciones aprendidas, de lo crudo a lo abstracto.
- 2 **Aproximación universal** garantiza capacidad, no generalización: la curva U manda, y el kit es conocido — weight decay (Ridge), early stopping (la U en vivo), dropout (ensamble implícito), batch norm, más datos.
- 3 Entrenar = **SGD con mini-batches** (Adam como default) sobre una pérdida no convexa que en la práctica se deja optimizar.
- 4 **Retropropagación** = regla de la cadena con reutilización:  $\delta^{(2)} = \hat{y} - y$ ;  $\delta^{(1)} = (w^{(2)}\delta^{(2)}) \odot g'(z)$ ; gradiente al costo de ida + vuelta. (Taller 2.)
- 5 **CNN**: localidad + pesos compartidos = ver con pocos parámetros; jerarquía bordes  $\rightarrow$  partes  $\rightarrow$  objetos; **transfer learning** para usarlas con datos de economista.
- 6 **Satélites**: de luces nocturnas (Henderson et al.) a mapas de pobreza (Jean et al.): medición barata, actualizable — y siempre un proxy que hay que validar.



Preparación para la próxima clase...

# Para aprovechar al máximo la próxima sesión

## Lecturas obligatorias de esta semana:

- Goodfellow, Bengio & Courville (2016), *Deep Learning*, cap. 6 (secs. 6.1–6.5).  
Guía: hasta backprop; el cap. 7 (regularización) es el complemento natural.
- Jean et al. (2016), *Science*.  
Guía: es corto; concéntrense en la figura 1 y el diseño del transfer learning.

**Recomendadas:** Henderson, Storeygard & Weil (2012); LeCun, Bengio & Hinton (2015, *Nature*, el survey).

# Para aprovechar al máximo la próxima sesión

## Lecturas obligatorias de esta semana:

- Goodfellow, Bengio & Courville (2016), *Deep Learning*, cap. 6 (secs. 6.1–6.5).  
Guía: hasta backprop; el cap. 7 (regularización) es el complemento natural.
- Jean et al. (2016), *Science*.  
Guía: es corto; concéntrense en la figura 1 y el diseño del transfer learning.

**Recomendadas:** Henderson, Storeygard & Weil (2012); LeCun, Bengio & Hinton (2015, *Nature*, el survey).

**Próxima sesión — la última de contenido: texto como datos y LLMs.** Del conteo de palabras a los *embeddings*, la atención y los *transformers*: entrenaremos un GPT de juguete en vivo y haremos *fine-tuning* de un modelo abierto. **Logística:** traigan portátil con cuenta de Google activa (usaremos Colab con GPU gratuita); la sesión corre en **Python** con cuadernos preparados — la excepción anunciada a la regla “R primero”.

# Para aprovechar al máximo la próxima sesión

## Lecturas obligatorias de esta semana:

- Goodfellow, Bengio & Courville (2016), *Deep Learning*, cap. 6 (secs. 6.1–6.5).  
Guía: hasta backprop; el cap. 7 (regularización) es el complemento natural.
- Jean et al. (2016), *Science*.  
Guía: es corto; concéntrense en la figura 1 y el diseño del transfer learning.

**Recomendadas:** Henderson, Storeygard & Weil (2012); LeCun, Bengio & Hinton (2015, *Nature*, el survey).

**Próxima sesión — la última de contenido: texto como datos y LLMs.** Del conteo de palabras a los *embeddings*, la atención y los *transformers*: entrenaremos un GPT de juguete en vivo y haremos *fine-tuning* de un modelo abierto. **Logística:** traigan portátil con cuenta de Google activa (usaremos Colab con GPU gratuita); la sesión corre en **Python** con cuadernos preparados — la excepción anunciada a la regla “R primero”.

**Aplicación de esta semana en R:** red feedforward con `keras` para predecir consumo eléctrico municipal, y clasificación de imágenes con una CNN preentrenada.

# Referencias esenciales

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Caps. 6–9.
- Cybenko, G. (1989). Approximation by Superpositions of a Sigmoidal Function. *Math. of Control, Signals and Systems*, 2, 303–314.
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proc. IEEE*, 86(11).
- Krizhevsky, A., Sutskever, I., & Hinton, G. (2012). ImageNet Classification with Deep Convolutional Neural Networks. *NeurIPS*.
- Srivastava, N., et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *JMLR*, 15, 1929–1958.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep Learning. *Nature*, 521, 436–444.
- Henderson, V., Storeygard, A., & Weil, D. (2012). Measuring Economic Growth from Outer Space. *AER*, 102(2), 994–1028.
- Jean, N., Burke, M., Xie, M., Davis, W., Lobell, D., & Ermon, S. (2016). Combining Satellite Imagery and Machine Learning to Predict Poverty. *Science*, 353(6301), 790–794.
- Dell, M. (2025). Deep Learning for Economists. *Journal of Economic Literature*, 63(1), 5–58.